

Recurso 1 — Mapa de decisiones para escoger tipos de <input>

Objetivo. Guiarte, paso a paso, para elegir el tipo de campo HTML5 correcto y su configuración mínima accesible (atributos, autocomplete, inputmode, ayudas y errores). Es una guía, no una actividad.

Cómo usar este recurso (3 pasos)

1. Identifica el dato que necesitas (correo, teléfono, número, fecha, archivo, texto...).
2. Sigue el diagrama (abajo) o la versión solo texto para escoger el tipo nativo.
3. Aplica la configuración recomendada (atributos, tokens de autocomplete, mensajes claros).

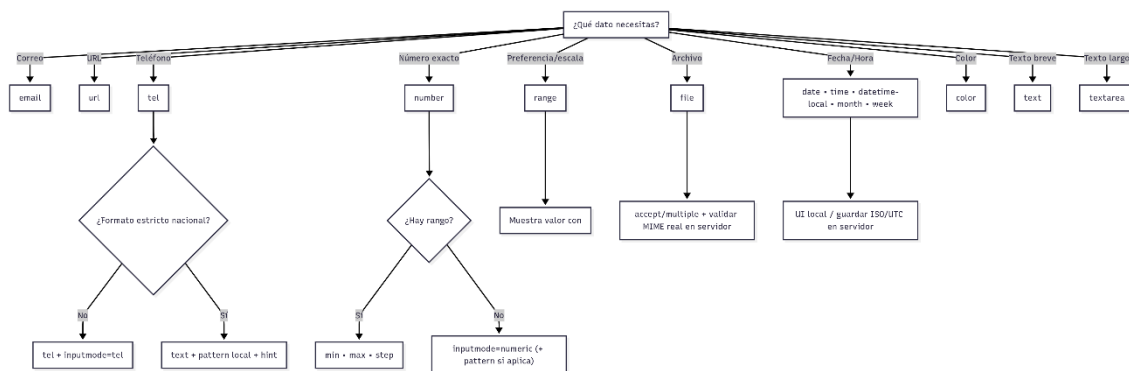


Figura 1. *Árbol de decisión para seleccionar el tipo de control HTML5* (Elaboración propia del autor, Damián Nicolalde, 2025)

- Correo → type="email"
- URL → type="url"
- Teléfono → type="tel" (si tu país exige formato rígido, usa type="text" + pattern + pista clara)
- Número exacto → type="number" (+ min/max/step si hay rango; si no, inputmode="numeric")
- Preferencia/escala → type="range" (+ <output> que muestre el valor)
- Fecha/hora → date / time / datetime-local / month / week (UI local; guardar en ISO/UTC)
- Archivo → type="file" (accept, multiple; en servidor validar MIME real y tamaño)
- Color → type="color"
- Texto breve → type="text" (+ minlength/maxlength)
- Texto largo → <textarea>

Tabla 1. *Tipo recomendado, atributos clave y sugerencias de A11y/UX* (Elaboración propia del autor, Damián Nicolalde, 2025)

Dato	Tipo recomendado	Atributos clave	Sugerencias A11y/UX
Correo	email	required	autocomplete="email", mensaje claro al fallar
URL	url	(opc.) required	Agrega ejemplo en placeholder (p. ej., https://...)
Teléfono	tel	—	inputmode="tel"; si hay formato rígido, usar pattern + hint
Número exacto	number	min max step	Evita bloquear mientras escribe; usa :invalid/:valid
Preferencia/escala	range	min max step	Muestra el valor con <output>
Fecha/Hora	date/time/datetime- local	—	UI local; persiste ISO/UTC en servidor
Archivo	file	accept multiple	Validar tipo real y tamaño en servidor
Color	color	—	Útil en personalización sencilla
Texto breve	text	minlength maxlength	inputmode según caso (numeric, email, etc.)
Texto largo	textarea	rows/cols	Evita validaciones intrusivas al teclear

Snippets mínimos por caso

Todos los ejemplos incluyen label, ayuda/error vinculados y comentarios.

Correo

```
<label for="e">Correo</label>
<input id="e" name="e" type="email" required
  autocomplete="email" aria-describedby="e-hint e-err">
<small id="e-hint">Usa tu correo institucional.</small>
<span id="e-err" role="alert" hidden>Formato: usuario@dominio.</span>
```

Teléfono (flexible)

```
<label for="t">Teléfono</label>
<input id="t" name="t" type="tel" inputmode="tel"
  aria-describedby="t-hint">
<small id="t-hint">Incluye código de área si aplica.</small>
```

Teléfono (formato estricto local — ejemplo)

```
<label for="t2">Teléfono (EC)</label>
<input id="t2" name="t2" type="text" inputmode="tel"
  pattern="^\+593\s?\d{2}\s?\d{3}\s?\d{4}$"
  aria-describedby="t2-hint t2-err">
<small id="t2-hint">Ej.: +593 22 345 6789</small>
<span id="t2-err" role="alert" hidden>Usa el formato del ejemplo.</span>
```

Número exacto (con rango)

```
<label for="n">Créditos</label>
<input id="n" name="n" type="number" min="1" max="40" step="1" required>
```

Escala/preferencia (mostrar valor)

```
<label for="s">Satisfacción</label>
<input id="s" type="range" min="1" max="5" step="1"
  oninput="s_out.value = this.value" value="3" aria-describedby="s-hint">
<small id="s-hint">1 = bajo, 5 = alto</small>
<output id="s_out">3</output>
```

Fecha (UI local; guardar ISO/UTC en servidor)

```
<label for="f">Fecha del turno</label>
<input id="f" name="f" type="date" required>
```

Archivo (validar en servidor tipo real y tamaño)

```
<label for="cv">CV (PDF)</label>
<input id="cv" name="cv" type="file" accept="application/pdf">
```

URL

```
<label for="u">Sitio web</label>
<input id="u" name="u" type="url" placeholder="https://ejemplo.edu.ec">
```

Texto breve

```
<label for="nick">Alias</label>
<input id="nick" name="nick" type="text" minlength="3" maxlength="20"
  aria-describedby="nick-hint">
<small id="nick-hint">3–20 caracteres, sin espacios.</small>
```

Texto largo

```
<label for="bio">Presentación</label>
<textarea id="bio" name="bio" rows="4"
  aria-describedby="bio-hint"></textarea>
<small id="bio-hint">Cuenta brevemente sobre ti (máx. 600 caracteres).</small>
```

Plantilla “campo bien formado” (rellena y reutiliza)

```
<label for="X">Etiqueta clara</label>
<input id="X" name="X" type="TIPO"
  autocomplete="TOKEN" inputmode="MODOS"
  required minlength="N" maxlength="M" min="A" max="B" step="S"
  aria-describedby="X-hint X-err">
<small id="X-hint">Pista breve (regla o ejemplo).</small>
<span id="X-err" role="alert" hidden>Mensaje de corrección.</span>
```

Notas clave

- **label for** define el nombre accesible del control.
- **aria-describedby** enlaza ayuda y error para que el lector los anuncie junto al campo.
- **Nativo primero**: si existe un tipo (email, url, date, number...), úsalo antes que pattern.
- **Móvil**: usa inputmode para mejorar el teclado (numeric, tel, email).

- **Archivos:** accept orienta, pero valida en servidor el MIME real y el tamaño.
- **Fechas:** UI local, persistencia ISO/UTC.

Recurso 2 — Biblioteca de mensajes accesibles de error (plantilla)

Objetivo. Proveer un patrón reutilizable (HTML+CSS+JS) para mostrar mensajes claros y anunciabiles, basado en la API nativa de validación, con extensiones para reglas cruzadas y validación asíncrona.

Patrón base (HTML + CSS)

```
<form id="f1" novalidate aria-labelledby="t-form">
  <h2 id="t-form">Registro</h2>

  <div class="field">
    <label for="email">Correo institucional</label>
    <input id="email" name="email" type="email" required
      autocomplete="email" aria-describedby="email-hint email-err">
    <small id="email-hint">Formato: usuario@dominio.</small>
    <span id="email-err" class="error" role="alert" hidden></span>
  </div>

  <div class="field">
    <label for="pass">Contraseña</label>
    <input id="pass" name="pass" type="password" required minlength="8"
      aria-describedby="pass-hint pass-err">
    <small id="pass-hint">Mín. 8 caracteres.</small>
    <span id="pass-err" class="error" role="alert" hidden></span>
  </div>

  <button type="submit">Crear cuenta</button>
</form>

/* Visibilidad de estados y foco */
.field input:invalid { border:2px solid #c00; }
.field input:valid { border:2px solid #0a0; }
:focus-visible { outline:3px solid #000; outline-offset:2px; }

/* Mensajes de error */
.error[hidden]{ display:none; }
.error{ display:block; margin-top:.25rem; }

/* No dependas solo del color: mantén el texto claro */
```

Maapeo de `ValidityState` → mensaje recomendado

```
const messages = {
  valueMissing: 'Completa este campo.',
  typeMismatch: 'Revisa el formato (p. ej., usuario@dominio).',
  patternMismatch: 'Sigue el formato indicado.',
  tooShort: (el) => `Añade ${el.minLength - el.value.length} caracteres más.`,
  tooLong: (el) => `Reduce a ${el.maxLength} caracteres.`,
  rangeUnderflow: (el) => `El valor mínimo es ${el.min}.`,
  rangeOverflow: (el) => `El valor máximo es ${el.max}.`,
  stepMismatch: 'Ajusta al paso permitido.',
  badInput: 'Usa solo números en este campo.',
  customError: (el) => el.validationMessage || 'Revisa este dato.'
};
```

Utilitario JS para un formulario (plug-and-play)

```

function attachA11yValidation(form){
  const fields = form.querySelectorAll('input, select, textarea');

  function showError(el, msg){
    const errId = (el.getAttribute('aria-describedby') || '').split(' ')
      .find(id => id && id.endsWith('-err'));
    const errEl = errId ? document.getElementById(errId) : null;
    el.setCustomValidity(msg || '');
    if (msg){
      el.setAttribute('aria-invalid', 'true');
      if (errEl){ errEl.textContent = msg; errEl.hidden = false; }
    } else {
      el.removeAttribute('aria-invalid');
      if (errEl){ errEl.hidden = true; errEl.textContent=''; }
    }
  }

  function nativeMessage(el){
    const v = el.validity;
    if (v.valid) return '';
    for (const key of Object.keys(messages)){
      if (v[key]){
        const m = messages[key];
        return typeof m === 'function' ? m(el) : m;
      }
    }
    return 'Revisa este dato.';
  }

  fields.forEach(el=>{
    el.addEventListener('input', ()=> showError(el, nativeMessage(el)));
    el.addEventListener('blur', ()=> showError(el, nativeMessage(el)));
  });

  form.addEventListener('submit', (e)=>{
    // Recorre y muestra el primer error
    for (const el of fields){
      const msg = nativeMessage(el);
      if (msg){
        e.preventDefault();
        showError(el, msg);
        el.focus(); // foco al primer campo con error
        break;
      }
    }
  });
}

// Activación
attachA11yValidation(document.getElementById('f1'));

```

Reglas cruzadas (ejemplo: “fin ≥ inicio”)

```
<label for="ini">Inicio</label>
<input id="ini" type="date" required>
<label for="fin">Fin</label>
<input id="fin" type="date" required aria-describedby="fin-err">
<span id="fin-err" role="alert" hidden></span>

<script>
const ini = document.getElementById('ini');
const fin = document.getElementById('fin');
function cross(){
  fin.setCustomValidity("");
  if (ini.value && fin.value && fin.value < ini.value){
    fin.setCustomValidity('La fecha de fin debe ser posterior a inicio. ');
  }
  fin.dispatchEvent(new Event('input')); // para refrescar mensaje con utilitario
}
ini.addEventListener('change', cross);
fin.addEventListener('change', cross);
</script>
```

Validación asíncrona (esqueleto con role="status")

```
<input id="mail2" type="email" required aria-describedby="st-mail2 mail2-err">
<small id="st-mail2" role="status" aria-live="polite"></small>
<span id="mail2-err" role="alert" hidden></span>

<script>
let debounce;
mail2.addEventListener('input', ()=>{
  clearTimeout(debounce);
  st-mail2.textContent = "";
  mail2.setCustomValidity("");
  if (!mail2.checkValidity()) return;

  debounce = setTimeout(async ()=>{
    st-mail2.textContent = 'Comprobando...';
    // const {exists} = await
    fetch('/api/check?e='+encodeURIComponent(mail2.value)).then(r=>r.json());
    const exists=false; // simula
    st-mail2.textContent = exists ? 'Este correo ya tiene cuenta.' : 'Disponible.';
    if (exists){ mail2.setCustomValidity('Correo ya registrado. '); }
    mail2.dispatchEvent(new Event('input'));
  }, 300);
});
</script>
```

Buenas prácticas de redacción (mensajes): “regla + cómo corrijo”, lenguaje simple, sin mayúsculas “a gritos”, sin tecnicismos innecesarios.

Recurso 3 — Checklist WCAG AA para formularios

Objetivo. Hoja de verificación operativa para revisar rápidamente formularios conforme a POUR (AA) antes de publicar.

Tabla 2. *Checklist por principio, con verificación y nota* (Elaboración propia del autor, Damián Nicolalde, 2025)

Ítem (POUR)	Cómo verificar	OK	N/A	Notas
P: Cada imagen informativa tiene alt útil; si requiere contexto, figure+figcaption.	Inspección + lector de pantalla.	☐	☐	
P: Contraste de texto $\geq 4.5:1$ (normal) / $3:1$ (grande).	Lighthouse/Axe.	☐	☐	
P: Zoom 200–400% sin romper (sin scroll horizontal general).	Zoom del navegador.	☐	☐	
O: Se puede completar todo con teclado (Tab/Shift+Tab/Enter/Espacio).	Recorrido manual.	☐	☐	
O: Foco visible y orden lógico.	Estilos :focus-visible.	☐	☐	
O: Modales accesibles (foco atrapado legítimo, cerrar con Esc, foco vuelve).	Prueba manual.	☐	☐	
U: Todos los campos tienen label for .	Inspección.	☐	☐	
U: Ayudas/errores vinculados (aria-describedby) y anunciables (role="alert"/aria-live).	Prueba de error.	☐	☐	
U: Jerarquía de encabezados correcta (h1→h2→h3).	Vista de encabezados.	☐	☐	
R: Semántica nativa; ARIA solo si falta (sin roles redundantes).	Validador + DevTools.	☐	☐	
R: Validación nativa encendida; reglas JS claras; servidor revalida.	Prueba checkValidity() + envío real.	☐	☐	

Prueba rápida

1. **Teclado:** completa el formulario sin mouse; confirma que el foco se ve en todo.
2. **Errores:** fuerza 2–3 errores; verifica que se lean y expliquen cómo corregir.
3. **Zoom 200%:** comprueba que no se rompa el layout.
4. **Lighthouse A11y:** ≥ 90 como barómetro interno (no sustituye revisión manual).

Prueba extendida (15 minutos)

- **Contraste** en componentes clave (botones, errores, links).
- **Lectores** (si es posible): NVDA (Win) o VoiceOver (Mac) en 1 flujo.
- **Validación asíncrona:** un caso real (p. ej., “correo ya registrado”) con role="status" y sin bloqueos.

Criterios de aceptación (pasa/no pasa)

- Al menos **90** en Lighthouse (Accesibility) + todos los ítems críticos **OK** (label, ayudas vinculadas, foco visible, teclado completo, zoom 200%).
- Sin **tablas para maquetar**; sin **placeholders como etiquetas**; sin quitar el **outline**.

